SIS 3204Y(5) - NLP

Last-Minute Revision Notes | Weeks 1-8 | Topic-wise

WEEK 1 - Introduction to NLP

Basics

- **NLP**: AI branch; computers understand, interpret, generate human language (text/speech).
- **Applications**: chatbots, machine translation, spam filtering, sentiment analysis, search engines, summarisation, QA.

Why language is hard

- **Ambiguity** (bank = river/finance)
- **Context dependence**
- **Sarcasm / irony**
- **Idioms** (break a leg)
- **Slang, typos, dialects**
- **Emotion, culture**

Traditional vs Modern NLP

Traditional	Modern
Handcrafted rules	Data-driven, learns from examples
Small, task-specific data	Massive text data
Separate system per task	General-purpose, multi-task
Limited context	Long-range context
Poor scaling	Scales with data + compute

NLP pipeline

- Input -> Text processing (cleaning, tokenisation) -> Representation (embeddings) -> Model -> Prediction/Inference -> Evaluation -> Output.

Evolution of NLP

- **Rule-based** (1950s-80s) -> **Statistical** (1990s, N-grams) -> **Machine Learning** (2000s, Naive Bayes) -> **Deep Learning** (RNN, LSTM) -> **Word embeddings** (Word2Vec) -> **Transformers** (2017, self-attention) -> **LLMs** (2020+).

Key terms

- **Tokenisation**: split text into tokens. Ex: "I love NLP" -> [I, love, NLP].
- **Word embedding**: dense real-valued vector; similar words are close (king ~ queen).
- **Transformer**: self-attention, parallel processing, long-range dependencies; base of BERT/GPT/LLMs.
- **LLM**: deep model trained on massive text (ChatGPT, Claude).
- **RAG**: retrieve external documents -> give as context to generator. Grounded answers, fewer hallucinations.
- **Hallucination**: fluent but false/fabricated output.
- **Ethics**: bias and fairness, privacy and security, misinformation.
- **Benefits of LLMs**: accuracy, versatility, natural generation. **Challenges**: hallucination, bias, compute cost, reasoning gaps.

WEEK 2 - Text Processing

- **Text processing**: clean, transform, represent raw text so machines can analyse it.
- **Messy text**: inconsistent case, punctuation, numbers, URLs/emails/mentions, spelling errors/slang, extra spaces.

Pipeline (6 stages)

- Cleaning -> Tokenisation -> Stop word removal -> Stemming/Lemmatisation -> Feature extraction -> Model-ready.

Key techniques

- **Lowercasing**: NLP/Nlp/nlp -> nlp. Reduces vocabulary. Bad for **NER**.
- **Text cleaning tasks**: lowercase, remove punctuation, numbers, URLs, HTML, hashtags, emojis, extra spaces.
- **Spelling normalisation** (SymSpell), **expand contractions** (don't -> do not; do before tokenisation, else "don"+"t"), **slang mapping** (u -> you).
- **Stop words**: the, is, at, of. Remove -> smaller vocabulary, faster, focus on content words. Risk: lose negation/question words ("not happy" -> "happy"; "where").
- Keep stop words for: sentiment, NLI, QA, Transformers.

Tokenisation

- **Types**: word, subword, character, sentence.
- **Methods**: whitespace, word, subword (BPE, WordPiece), character.

- **Subword**: internationalization -> inter + ##national + ##ization. Solves **OOV**, smaller vocabulary, used in GPT/LLaMA/Gemini.
- **Special tokens**: [CLS], [SEP].
- **Sentence tokenisation challenges**: abbreviations (Dr., U.S.A.), ellipses, ?/!, newlines, domain style.

Approach	Pros / Cons
Word-level	Simple; fails on OOV
Subword (BPE)	Handles any word; fewer tokens; used in LLMs
Character-level	No OOV; too many tokens, slow

Stemming vs Lemmatisation

Criterion	Stemming	Lemmatisation
Method	Rule-based prefix/suffix chop	Vocabulary + morphology + POS
Output	May be non-word (studi)	Valid lemma (study)
Speed	Faster	Slower
Accuracy	Lower	Higher
Use	Quick tasks	Search, QA, summarisation

- Ex: mice -> mouse, went -> go, better -> good (needs POS).

Evaluation metrics of preprocessing

- **Token count reduction** = (1 - New/Original) x 100.
- **Unique token ratio** = unique/total.
- **Information retention**, **stop word %**, **processing time**.

Best practice

- **No one-size-fits-all**: match steps to task + data + model.
- Start simple, inspect outputs, same pipeline on train/val/test, keep raw copy, avoid over-processing.

WEEK 3 - Feature Extraction and Text Encoding

- **Feature extraction**: raw text -> numerical vectors. Needed because models need numbers, capture useful info, reduce noise, help generalisation.
- **Why words cannot go directly**: need numbers; no numeric meaning; dimensionality mismatch; no generalisation to unseen words.

Representation pipeline

- Raw text -> Preprocess -> Tokenise -> Build vocabulary -> Index -> Encode -> Feature vectors.
- **Vocabulary**: unique tokens with index. Ex: the=0, cat=1, sat=2, on=3, mat=4.

Integer (label) encoding

- Each token = one integer. Simple, memory efficient.
- **Problems**: false order, false distance, no semantics, sensitive to vocabulary change.

One-hot encoding

- Binary vector: single 1 at word index. Matrix shape = V x N.
- **Pros**: no false order/distance; equal treatment; works with linear models/NN.
- **Cons**: high-dimensional, **sparse**, no meaning (all vectors **orthogonal**), costly, no frequency info.
- **Sparse problem**: V=100,000; 1M vectors = 100 billion entries; 99.999% zeros.

Sparse vs Dense

Aspect	Sparse (One-hot, BoW, TF-IDF)	Dense (Word2Vec, GloVe, BERT)
Size	Very high (=V)	Low (50-500)
Values	Mostly 0	Mostly non-zero, real
Meaning	No	Yes
Similarity	None	Similar words close
Memory	High	Low

- **BoW**: counts, ignores order. **TF-IDF**: word importance; still sparse, no meaning.

Word embeddings and Word2Vec

- **Embeddings**: dense vectors; capture meaning + relations (king - man + woman ~ queen); better generalisation.
- **Word2Vec**: unsupervised, Mikolov 2013. Based on **distributional hypothesis**: words in similar contexts have similar meaning.
- Slide **context window** -> (target, context) pairs -> train with SGD.

Aspect	CBOW	Skip-Gram
Direction	Context -> Target	Target -> Context
Input / Output	Many context / one target	One target / many context
Best for	Frequent words	Rare words
Speed	Faster	Slower
Data	Less data OK	Needs more data

CBOW context vector = average of context vectors. Ex: (v_the + v_cat) / 2.

Word2Vec limitations -> Fix

- Local context only; CBOW ignores order; no polysemy (bank); OOV words; static embeddings; linear relations only.
- **Fix**: Transformers/attention, contextual embeddings (ELMo, BERT, GPT), subword tokenisation.
- **Chain**: Integer -> One-hot -> BoW/TF-IDF -> Word2Vec -> Contextual embeddings.

WEEK 4 - Neural Networks for NLP

Components

- **ANN**: layers of connected neurons. **Layers**: input (features), hidden (learned patterns), output (prediction).
- **Neuron**: inputs, **weights** (importance), **bias** (shift) -> weighted sum -> activation.

z = Sum(w_i x_i) + b | output = activation(z)

Activation functions

- **Purpose**: add non-linearity.
- **ReLU**: max(0,z). **Sigmoid**: output 0-1 (binary probability). **Softmax**: multi-class probabilities, sum = 1.

Training cycle

- **Forward propagation** -> Prediction -> **Loss** -> **Backpropagation** (gradients) -> **Gradient descent** -> Weight update.
- **Loss**: distance between prediction and true label. Cross-entropy: L = -ln(p_correct). Ex: -ln(0.8) = 0.223.
- **Gradient**: direction/effect of parameter on loss.
- **Learning rate**: step size.

w_new = w_old - (learning rate x gradient) Ex: 0.50 - (0.10 x 0.40) = 0.46

Terms

- **Epoch**: one full pass of data. **Batch**: subset per update. 1000 samples / batch 100 = 10 updates/epoch; 20 epochs = 200 updates.
- **Train**: learn parameters. **Validation**: tuning, early stopping. **Test**: final unbiased evaluation (use once).

Fit problems

- **Underfitting**: too simple/undertrained.
- **Overfitting**: high train accuracy, low val/test accuracy (poor generalisation).
- **Remedies**: dropout, early stopping, more data, simpler model, L1/L2 regularisation, data augmentation.
- **Dropout**: randomly disable neurons in training -> less co-adaptation -> regularisation.

Text classification pipeline

- Text -> Tokenise -> Embeddings -> Network input -> Forward pass -> Softmax -> Highest probability = class.
- Ex: Positive 0.72, Neutral 0.18, Negative 0.10 -> **Positive**.

WEEK 5 - RNN, LSTM, GRU

RNN

- Processes tokens **sequentially**; embedding + previous **hidden state** -> new hidden state.
- **Hidden state** = memory of previous words; final hidden state -> prediction (e.g. sentiment).
- **Classification**: tokenise -> embed -> RNN -> final hidden state -> softmax.
- **BPTT**: unroll through time; forward pass -> loss -> propagate error backwards across time steps -> gradients -> update shared weights.

Vanishing gradient

- Gradients shrink over many time steps in BPTT.
- Early words get tiny updates -> RNN fails at **long-term dependencies**. Ex: "The cat that chased the mouse **was** hungry".

LSTM

- Has **cell state** (long-term memory) + **hidden state** (current output).
- **Forget gate**: what to discard from old cell state (0 = forget, 1 = keep).
- **Input gate**: what new info to add.
- **Output gate**: what part of cell state goes to hidden state.

- **Flow**: input + h(t-1) + c(t-1) -> forget -> input -> new cell state -> output gate -> new hidden state.

GRU

- No separate cell state. Two gates:
- **Reset gate**: how much past to use for new candidate (low = ignore past).
- **Update gate**: balance between keeping old state and using new candidate.

Feature	RNN	LSTM	GRU
States	Hidden	Hidden + cell	Hidden
Gates	None	3 (forget, input, output)	2 (reset, update)
Long-term memory	Poor	Good	Good
Complexity	Lowest	Highest	Medium (fewer params)

- **Similarities of LSTM/GRU**: both gated; both control past vs new info; fix vanishing gradient partly.

Architectures and applications

- **Many-to-one**: sequence -> single output (sentiment).
- **Many-to-many**: sequence -> sequence (NER, POS tagging, translation).
- Applications: language modelling, text classification, NER, machine translation, summarisation, chatbots.

Limits of recurrence -> Attention

- **Sequential** processing, no parallelisation, slow on long text; LSTM/GRU improve but don't remove this.
- **Attention** gives direct access to relevant positions; **Transformers** remove recurrence -> parallel, scalable.
- **Chain**: Feed-forward NN (no memory) -> RNN (hidden state) -> LSTM/GRU (gates) -> Attention/Transformer.

WEEK 6 - Attention and Self-Attention

Why attention

- **Encoder-decoder bottleneck**: one fixed context vector loses info on long input.
- **Attention**: decoder looks at all encoder hidden states; context changes at each decoding step.
- Ex: generating "chat" attends to "cat"; "assis" attends to "sat".

Score -> Weight -> Context

- **Attention score**: relevance of an input to the query.
- **Attention weight**: softmax of scores; positive, sum = 1.
- **Context vector**: weighted sum of hidden/Value vectors.
- **Why softmax**: turns raw scores into relative contributions.

Context = w1 h1 + w2 h2 + w3 h3 Ex: [0.5,0.3,0.2] with h1=[2,0], h2=[0,1], h3=[1,1] -> [1.2, 0.5]

Dot product: q=[1,1], k=[1,0] -> (1x1)+(1x0) = 1. Highest score = most relevant key.

Self-attention

- Each token attends to other tokens in the **same sequence** -> contextual representation.
- **Encoder-decoder attention**: decoder query attends to encoder states. **Self-attention**: Q, K, V from same sequence.
- Captures: local syntax, long-range dependency, pronoun reference, semantic context (bank + loan).

Query, Key, Value

- **Query**: what token is looking for. **Key**: used to match with query. **Value**: information that is combined.
- **Learned projections**: $Q = XW_Q$, $K = XW_K$, $V = XW_V$.
- Misconception: Q, K, V come from same X but use **different weight matrices** and roles.

Scaled dot-product attention

$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \text{sqrt}(d_k)) V$

- **QK^T**: relevance scores.
- **Divide by sqrt(d_k)**: large dot products make softmax too peaked and gradients small; scaling stabilises training.
- **Softmax**: weights. **x V**: contextual output.

Step-by-step

- Input embeddings -> create Q,K,V -> QK^T -> scale -> softmax -> weights x V -> contextual output.

Multi-head attention

- Several attention heads in **parallel**, each with own Q/K/V projections.
- Outputs **concatenated** -> **output projection**.
- Benefit: multiple patterns/**representation subspaces**. Caution: a head need not match one linguistic rule.

Bridge to Transformer

- Attention -> Self-attention -> Q/K/V -> Scaled dot-product -> Multi-head -> **Positional information** + feed-forward -> Transformer blocks.

WEEK 7 - Transformer, BERT, GPT

Transformer basics

- **Fixes RNN limits**: sequential processing, weak long-range dependency.
- **Positional encoding**: self-attention has no order; needed ("dog bites man" vs "man bites dog").
- **Token embedding**: identity/meaning. **Positional embedding**: position.
- **Parallel processing**: faster training.

Encoder layer components

- **Multi-head self-attention**: heads capture different relations.
- **Position-wise FFN**: same two linear layers + non-linearity applied to each token independently.
- **Residual connections**: preserve information, improve gradient flow.
- **Layer normalisation**: stabilises training.

BERT

- **Encoder-only**; **bidirectional** context (left + right).
- Pretraining: **Masked Language Modelling** (predict [MASK]).
- Classification: add **classification head** on [CLS], **fine-tune** on labelled data.
- Best for: NER, classification, understanding tasks.

GPT

- **Decoder-only**; **autoregressive**; **causal (masked) self-attention** blocks future tokens.
- Generation loop: predict next token -> append to context -> repeat until stop.
- **Greedy decoding**: pick highest probability. **Sampling**: draw from distribution, can pick other tokens.
- **Base GPT**: next-token prediction. **Instruction tuning**: trained on instruction-response pairs to follow requests.
- **Training**: parameters updated by minimising loss. **Inference**: parameters fixed.

Which architecture

Task	Architecture	Reason
NER, classification	Encoder-only (BERT)	Bidirectional representation
Text generation, reply	Decoder-only (GPT)	Autoregressive generation
Translation	Encoder-decoder (T5, Marian)	Encoder reads source; decoder generates with **cross-attention**

WEEK 8 - Foundation Models, LLMs, Prompting, DeepEval

Foundation models and LLMs

- **Foundation model**: broad large-scale pre-training; adaptable to many downstream tasks.
- **Not all are LLMs**: can be vision/audio models. LLM = language-focused.
- **Pre-training**: learns statistical patterns from huge text (next-token prediction).
- **Limitations**: hallucination, bias, outdated knowledge; fluent does not mean reliable.

Prompt engineering

- **Weak prompt**: no labels, no output format, no rule for mixed/missing cases.
- **Good prompt**: clear task + allowed labels + rule for edge cases + output format (e.g. JSON, "return only label") + missing value rule (null).
- **Zero-shot**: instruction only. **Few-shot**: includes labelled examples (**in-context learning**).
- Few-shot structure: instruction -> 2 examples (Message / Label) -> new message with blank Label.

LLM evaluation (DeepEval)

- **DeepEval**: framework to test LLM outputs with test cases and metrics.
- **Answer Relevancy**: does the response address the question?
- **Faithfulness**: are claims supported by retrieved context?
- Relevant does not mean correct. A wrong but on-topic answer: relevancy high, faithfulness low.

Case-study pattern

- Compare chatbot answer with retrieved document -> list contradictions (wrong date, wrong channel, wrong penalty).
- **Improve reliability**: ground answers in retrieved sources (RAG), abstain/escalate if no support, automated faithfulness tests, human review of flagged answers.

QUICK EXAM CHECKLIST

- **Formulas**: z = Sum(wx)+b; w_new = w - lr x grad; L = -ln(p); Attention = softmax($QK^T/sqrt(d_k)$)V; context = weighted sum.
- **Always mention**: hidden state, gates, cell state, vanishing gradient, Q/K/V, softmax, positional encoding.
- **Comparison questions**: use a table, 4-6 aspects.

- **Calculation questions**: write formula -> substitute -> final answer.
- **Chains to memorise**: representation evolution; sequence-model evolution; attention-to-Transformer bridge; training cycle.